



Buatkan saya Authentication seperti perusahaan enterpraise , seperti Gojek , Grab , Tokped dan perbankan , menggunakan tech Go , Gin , Gorm
-Struktur project, dependency injection, konfigurasi JWT, ▼

Berikut adalah arsitektur **Enterprise Authentication & Authorization System** yang telah dirancang secara khusus untuk disesuaikan dengan struktur folder project Anda (**Clean/Hexagonal Architecture**).

Sistem ini menerapkan standar keamanan kelas enterprise (mirip Gojek, Grab, Tokopedia, & Bank) yang mencakup:

- **Refresh Token Rotation & Invalidation**
- **Device/Session Management** (Logout & Logout All Devices)
- **Account Lockout Protection** (Anti-Brute Force)
- **Audit Logging** (Pencatatan aktivitas login & security events)
- **Role-Based Access Control (RBAC) Middleware**
- **Password Reset & Email Verification Flow**
- **Security Hardening & Rate Limiting**

1. Instalasi Dependency Kebutuhan

Jalankan perintah berikut di terminal project Anda:

Bash



```
go get -u github.com/golang-jwt/jwt/v5
go get -u golang.org/x/crypto/bcrypt
go get -u golang.org/x/time/rate
```

2. Updates & Entity Tambahan (internal/entities)

A. Update internal/entities/user.go

Tambahkan field keamanan pada entity User :

Go



```
package entities

import (
    "attendance-api/internal/enums"
    "time"
)

type User struct {
    Base
    FullName          string    `json:"fullName" gorm:"column:full_name;not`
    Email             string    `json:"email" gorm:"column:email;uniqueInde`
    PhoneNumber       string    `json:"phoneNumber" gorm:"column:phone_num`
    Password          string    `json:"- " gorm:"column:password;not null"`
    Role              enums.Role `json:"role" gorm:"column:role;type:varchar`
    LastLogin         *time.Time `json:"lastLogin" gorm:"column:last_login;d`
    IsActive          bool      `json:"isActive" gorm:"column:is_active;def`

    // Field Keamanan Enterprise
    IsEmailVerified    bool      `json:"isEmailVerified" gorm:"column:is_ema`
    EmailVerificationToken string    `json:"- " gorm:"column:email_verification_t`
    FailedLoginAttempts int       `json:"- " gorm:"column:failed_login_attempt`
    LockedUntil        *time.Time `json:"- " gorm:"column:locked_until;default`
    PasswordResetToken string    `json:"- " gorm:"column:password_reset_token`
    PasswordResetExpires *time.Time `json:"- " gorm:"column:password_reset_expir`
}
```

B. Entity Session internal/entities/user_session.go

Digunakan untuk melacak device login user, pembatalan token (token revocation), dan multi-device management.

Go



```
package entities

import (
```

```

        "time"
    )

    type UserSession struct {
        Base
        UserID      string    `json:"userId" gorm:"type:uuid;index;not null"`
        User         User      `json:"-" gorm:"foreignKey:UserID;constraint:OnDel
        RefreshTokenHash string    `json:"-" gorm:"column:refresh_token_hash;uniqueIn
        IPAddress    string    `json:"ipAddress" gorm:"column:ip_address"`
        UserAgent     string    `json:"userAgent" gorm:"column:user_agent"`
        IsRevoked     bool      `json:"isRevoked" gorm:"column:is_revoked;default:
        ExpiresAt    time.Time `json:"expiresAt" gorm:"column:expires_at;not null
    }

```

C. Entity Audit Log `internal/entities/audit_log.go`

Mencatat seluruh rekam jejak aktivitas sensitif sistem (kebutuhan audit perbankan/enterprise).

Go



```

package entities

type AuditLog struct {
    Base
    UserID    *string `json:"userId" gorm:"type:uuid;index"`
    Action    string  `json:"action" gorm:"type:varchar(100);not null" // e.g.,
    IPAddress string  `json:"ipAddress" gorm:"type:varchar(45)"`
    UserAgent string  `json:"userAgent" gorm:"type:text"`
    Details   string  `json:"details" gorm:"type:text"`
}

```

3. Package Security & Utils (`pkg/security` / `internal/utils`)

A. Utilities Password & Token (`internal/utils/security.go`)

Go



```

package utils

```

```

import (
    "crypto/rand"
    "crypto/sha256"
    "encoding/hex"
    "golang.org/x/crypto/bcrypt"
)

func HashPassword(password string) (string, error) {
    bytes, err := bcrypt.GenerateFromPassword([]byte(password), 12) // Cost 12 un
    return string(bytes), err
}

func CheckPasswordHash(password, hash string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
    return err == nil
}

func GenerateRandomToken() (string, error) {
    b := make([]byte, 32)
    _, err := rand.Read(b)
    if err != nil {
        return "", err
    }
    return hex.EncodeToString(b), nil
}

func HashToken(token string) string {
    h := sha256.New()
    h.Write([]byte(token))
    return hex.EncodeToString(h.Sum(nil))
}

```

B. JWT Helper (`internal/utils/jwt.go`)

```

Go

package utils

import (
    "attendance-api/internal/enums"
    "errors"
    "time"

    "github.com/golang-jwt/jwt/v5"
)

```

```

type JWTClaims struct {
    UserID      string    `json:"userId"`
    Role        enums.Role `json:"role"`
    SessionID   string    `json:"sessionId"`
    jwt.RegisteredClaims
}

func GenerateAccessToken(userID string, role enums.Role, sessionID string, secret string) (string, error) {
    claims := JWTClaims{
        UserID:      userID,
        Role:        role,
        SessionID:   sessionID,
        RegisteredClaims: jwt.RegisteredClaims{
            ExpiresAt: jwt.NewNumericDate(time.Now().Add(time.Duration(tt
            IssuedAt: jwt.NewNumericDate(time.Now()),
        },
    },

    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString([]byte(secret))
}

func ValidateToken(tokenStr string, secret string) (*JWTClaims, error) {
    token, err := jwt.ParseWithClaims(tokenStr, &JWTClaims{}, func(token *jwt.Token) (interface{}, error) {
        if _, ok := token.Method.(*jwt.SigningMethodHMAC); !ok {
            return nil, errors.New("method signing token tidak valid")
        }
        return []byte(secret), nil
    })

    if err != nil {
        return nil, err
    }

    claims, ok := token.Claims.(*JWTClaims)
    if !ok || !token.Valid {
        return nil, errors.New("token tidak valid")
    }

    return claims, nil
}

```

4. Repository Auth

(internal/adapters/repositories/auth_repository.
go)

```
package repositories

import (
    "attendance-api/internal/entities"
    "time"

    "gorm.io/gorm"
)

type AuthRepository interface {
    FindByEmail(email string) (*entities.User, error)
    FindByID(id string) (*entities.User, error)
    UpdateUser(user *entities.User) error

    // Session Management
    CreateSession(session *entities.UserSession) error
    FindSessionByHash(hash string) (*entities.UserSession, error)
    RevokeSessionByID(sessionID string) error
    RevokeAllUserSessions(userID string) error

    // Audit Logging
    CreateAuditLog(log *entities.AuditLog) error

    // Password Reset & Verification
    FindByResetToken(token string) (*entities.User, error)
    FindByVerificationToken(token string) (*entities.User, error)
}

type authRepository struct {
    db *gorm.DB
}

func NewAuthRepository(db *gorm.DB) AuthRepository {
    return &authRepository{db: db}
}

func (r *authRepository) FindByEmail(email string) (*entities.User, error) {
    var user entities.User
    err := r.db.Where("email = ?", email).First(&user).Error
    if err != nil {
        return nil, err
    }
    return &user, nil
}

func (r *authRepository) FindByID(id string) (*entities.User, error) {
```

```

        var user entities.User
        err := r.db.Where("id = ?", id).First(&user).Error
        if err != nil {
            return nil, err
        }
        return &user, nil
    }

    func (r *authRepository) UpdateUser(user *entities.User) error {
        return r.db.Save(user).Error
    }

    func (r *authRepository) CreateSession(session *entities.UserSession) error {
        return r.db.Create(session).Error
    }

    func (r *authRepository) FindSessionByHash(hash string) (*entities.UserSession, error) {
        var session entities.UserSession
        err := r.db.Where("refresh_token_hash = ?", hash).First(&session).Error
        if err != nil {
            return nil, err
        }
        return &session, nil
    }

    func (r *authRepository) RevokeSessionByID(sessionID string) error {
        return r.db.Model(&entities.UserSession{}).Where("id = ?", sessionID).Update(
    }

    func (r *authRepository) RevokeAllUserSessions(userID string) error {
        return r.db.Model(&entities.UserSession{}).Where("user_id = ?", userID).Update(
    }

    func (r *authRepository) CreateAuditLog(log *entities.AuditLog) error {
        return r.db.Create(log).Error
    }

    func (r *authRepository) FindByResetToken(token string) (*entities.User, error) {
        var user entities.User
        err := r.db.Where("password_reset_token = ?", token).First(&user).Error
        return &user, err
    }

    func (r *authRepository) FindByVerificationToken(token string) (*entities.User, error) {
        var user entities.User
        err := r.db.Where("email_verification_token = ?", token).First(&user).Error
        return &user, err
    }

```

5. Middleware Auth & RBAC

(internal/adapters/middleware)

A. Auth & RBAC Middleware

(internal/adapters/middleware/auth_middleware.go)

Go



```
package middleware

import (
    "attendance-api/config"
    "attendance-api/internal/adapters/repositories"
    "attendance-api/internal/enums"
    "attendance-api/internal/utils"
    "net/http"
    "strings"

    "github.com/gin-gonic/gin"
)

func AuthMiddleware(authRepo repositories.AuthRepository) gin.HandlerFunc {
    return func(c *gin.Context) {
        authHeader := c.GetHeader("Authorization")
        if authHeader == "" {
            c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
                return
            }

            tokenParts := strings.Split(authHeader, " ")
            if len(tokenParts) != 2 || tokenParts[0] != "Bearer" {
                c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
                return
            }

            secret := config.Config("JWT_SECRET")
            claims, err := utils.ValidateToken(tokenParts[1], secret)
            if err != nil {
                c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"error":
                return
            }

            // simpan context untuk handler selanjutnya
            c.Set("userID", claims.UserID)
            c.Set("userRole", claims.Role)
            c.Set("sessionID", claims.SessionID)
```



```

        c.Next()
    }
}

func RequireRoles(roles ...enums.Role) gin.HandlerFunc {
    return func(c *gin.Context) {
        roleVal, exists := c.Get("userRole")
        if !exists {
            c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "R
            return
        }

        userRole := roleVal.(enums.Role)
        hasRole := false
        for _, r := range roles {
            if r == userRole {
                hasRole = true
                break
            }
        }

        if !hasRole {
            c.AbortWithStatusJSON(http.StatusForbidden, gin.H{"error": "A
            return
        }

        c.Next()
    }
}

```

B. Rate Limiting Middleware

(`internal/adapters/middleware/rate_limit.go`)

Go



```

package middleware

import (
    "net/http"
    "sync"

    "github.com/gin-gonic/gin"
    "golang.org/x/time/rate"
)

```

```

var limiters = make(map[string]*rate.Limiter)
var mu sync.Mutex

func getVisitorLimiter(ip string) *rate.Limiter {
    mu.Lock()
    defer mu.Unlock()

    limiter, exists := limiters[ip]
    if !exists {
        // Izinkan maks 5 request per detik, burst hingga 10 request
        limiter = rate.NewLimiter(5, 10)
        limiters[ip] = limiter
    }

    return limiter
}

func RateLimiter() gin.HandlerFunc {
    return func(c *gin.Context) {
        ip := c.ClientIP()
        limiter := getVisitorLimiter(ip)

        if !limiter.Allow() {
            c.AbortWithStatusJSON(http.StatusTooManyRequests, gin.H{
                "error": "Maksimum rate limit terlampaui. Silakan cob
            })
            return
        }

        c.Next()
    }
}

```

6. Usecase Auth (internal/usecases/auth_usecase.go)

Go



```

package usecases

import (
    "attendance-api/config"
    "attendance-api/internal/adapters/repositories"
    "attendance-api/internal/entities"
    "attendance-api/internal/utls"
    "errors"
    "fmt"

```

```

        "strconv"
        "time"
    )

    type AuthUseCase interface {
        Login(email, password, ip, userAgent string) (accessToken, refreshToken string, err error)
        RefreshToken(refreshTokenStr, ip, userAgent string) (newAccessToken, newRefreshToken string, err error)
        Logout(sessionID string) error
        LogoutAll(userID string) error
        GetProfile(userID string) (*entities.User, error)
        ForgotPassword(email, ip, userAgent string) error
        ResetPassword(token, newPassword string) error
        VerifyEmail(token string) error
    }

    type authUseCase struct {
        authRepo repositories.AuthRepository
    }

    func NewAuthUseCase(authRepo repositories.AuthRepository) AuthUseCase {
        return &authUseCase{authRepo: authRepo}
    }

    func (u *authUseCase) Login(email, password, ip, userAgent string) (string, string, error) {
        user, err := u.authRepo.FindByEmail(email)
        if err != nil {
            u.logAudit(nil, "LOGIN_FAILED", ip, userAgent, fmt.Sprintf("Email tidak ditemukan"))
            return "", "", errors.New("email atau password salah")
        }

        // 1. Cek Apakah Akun Dikunci (Account Lock)
        if user.LockedUntil != nil && user.LockedUntil.After(time.Now()) {
            return "", "", fmt.Errorf("Akun Anda terkunci hingga %s karena terlalu banyak gagal login", user.LockedUntil.Format("2 Jan 2006 15:04:05"))
        }

        // 2. Verifikasi Password
        if !utils.CheckPasswordHash(password, user.Password) {
            user.FailedLoginAttempts++
            maxAttempts, _ := strconv.Atoi(config.Config("MAX_LOGIN_ATTEMPTS"))
            if maxAttempts == 0 {
                maxAttempts = 5
            }

            if user.FailedLoginAttempts >= maxAttempts {
                lockTime := time.Now().Add(15 * time.Minute)
                user.LockedUntil = &lockTime
                u.logAudit(&user.ID, "ACCOUNT_LOCKED", ip, userAgent, "Terlalu banyak gagal login, akun terkunci")
            }
        }
    }

```

```

        u.authRepo.UpdateUser(user)
        u.logAudit(&user.ID, "LOGIN_FAILED", ip, userAgent, "Password salah")
        return "", "", errors.New("email atau password salah")
    }

    // 3. Reset stats percobaan jika berhasil
    now := time.Now()
    user.FailedLoginAttempts = 0
    user.LockedUntil = nil
    user.LastLogin = &now
    _ = u.authRepo.UpdateUser(user)

    // 4. Buat Refresh Token & Session
    rawRefreshToken, _ := utils.GenerateRandomToken()
    refreshTokenHash := utils.HashToken(rawRefreshToken)

    ttlDays, _ := strconv.Atoi(config.Config("REFRESH_TOKEN_TTL_DAYS"))
    if ttlDays == 0 {
        ttlDays = 7
    }

    session := &entities.UserSession{
        UserID:         user.ID,
        RefreshTokenHash: refreshTokenHash,
        IPAddress:       ip,
        UserAgent:       userAgent,
        ExpiresAt:       time.Now().AddDate(0, 0, ttlDays),
    }

    if err := u.authRepo.CreateSession(session); err != nil {
        return "", "", err
    }

    // 5. Buat Access Token
    jwtTTL, _ := strconv.Atoi(config.Config("JWT_TTL_MINUTES"))
    if jwtTTL == 0 {
        jwtTTL = 15
    }

    accessToken, err := utils.GenerateAccessToken(user.ID, user.Role, session.ID,
    if err != nil {
        return "", "", err
    }

    u.logAudit(&user.ID, "LOGIN_SUCCESS", ip, userAgent, "Login Berhasil")
    return accessToken, rawRefreshToken, nil
}

```

```

func (u *authUseCase) RefreshToken(refreshTokenStr, ip, userAgent string) (string, string, error) {
    hash := utils.HashToken(refreshTokenStr)
    session, err := u.authRepo.FindSessionByHash(hash)

```

```

    if err != nil || session.IsRevoked || session.ExpiresAt.Before(time.Now()) {
        return "", "", errors.New("refresh token tidak valid atau telah kedaluwarsa")
    }

    // Refresh Token Rotation: Revoke token lama
    _ = u.authRepo.RevokeSessionByID(session.ID)

    // Ambil data User
    user, err := u.authRepo.FindByID(session.UserID)
    if err != nil || !user.IsActive {
        return "", "", errors.New("user tidak aktif atau tidak ditemukan")
    }

    // Buat Session Baru
    newRawRefreshToken, _ := utils.GenerateRandomToken()
    newHash := utils.HashToken(newRawRefreshToken)

    newSession := &entities.UserSession{
        UserID:          user.ID,
        RefreshTokenHash: newHash,
        IPAddress:       ip,
        UserAgent:       userAgent,
        ExpiresAt:       time.Now().AddDate(0, 0, 7),
    }
    _ = u.authRepo.CreateSession(newSession)

    // Issue Access Token Baru
    jwtTTL, _ := strconv.Atoi(config.Config("JWT_TTL_MINUTES"))
    newAccessToken, _ := utils.GenerateAccessToken(user.ID, user.Role, newSession)

    u.logAudit(&user.ID, "TOKEN_REFRESHED", ip, userAgent, "Token Diperbarui")
    return newAccessToken, newRawRefreshToken, nil
}

func (u *authUseCase) Logout(sessionID string) error {
    return u.authRepo.RevokeSessionByID(sessionID)
}

func (u *authUseCase) LogoutAll(userID string) error {
    return u.authRepo.RevokeAllUserSessions(userID)
}

func (u *authUseCase) GetProfile(userID string) (*entities.User, error) {
    return u.authRepo.FindByID(userID)
}

func (u *authUseCase) ForgotPassword(email, ip, userAgent string) error {
    user, err := u.authRepo.FindByEmail(email)
    if err != nil {

```

```

        return nil // Kembalikan response seolah-olah sukses untuk mencegah U
    }

    resetToken, _ := utils.GenerateRandomToken()
    expiresAt := time.Now().Add(30 * time.Minute)

    user.PasswordResetToken = utils.HashToken(resetToken)
    user.PasswordResetExpires = &expiresAt
    _ = u.authRepo.UpdateUser(user)

    u.logAudit(&user.ID, "FORGOT_PASSWORD_REQUESTED", ip, userAgent, "Minta reset

    // TODO: Panggil EmailService Anda di sini untuk mengirim resetToken
    // emailUseCase.SendEmail(user.Email, "Reset Password", "Token Anda: " + rese

    return nil
}

func (u *authUseCase) ResetPassword(rawToken, newPassword string) error {
    hashedToken := utils.HashToken(rawToken)
    user, err := u.authRepo.FindByResetToken(hashedToken)
    if err != nil || user.PasswordResetExpires == nil || user.PasswordResetExpire
        return errors.New("token reset password tidak valid atau kedaluwarsa")
    }

    hashedPassword, err := utils.HashPassword(newPassword)
    if err != nil {
        return err
    }

    user.Password = hashedPassword
    user.PasswordResetToken = ""
    user.PasswordResetExpires = nil
    _ = u.authRepo.UpdateUser(user)

    // Revoke semua sesi lama untuk keamanan penuh
    _ = u.authRepo.RevokeAllUserSessions(user.ID)

    u.logAudit(&user.ID, "PASSWORD_RESET_SUCCESS", "", "", "Password berhasil diu
    return nil
}

func (u *authUseCase) VerifyEmail(rawToken string) error {
    hashedToken := utils.HashToken(rawToken)
    user, err := u.authRepo.FindByVerificationToken(hashedToken)
    if err != nil {
        return errors.New("token verifikasi email tidak valid")
    }
}

```

```

        user.IsEmailVerified = true
        user.EmailVerificationToken = ""
        return u.authRepo.UpdateUser(user)
    }

    func (u *authUseCase) logAudit(userID *string, action, ip, userAgent, details string)
        _ = u.authRepo.CreateAuditLog(&entities.AuditLog{
            UserID:    userID,
            Action:    action,
            IPAddress: ip,
            UserAgent: userAgent,
            Details:   details,
        })
    }
}

```

7. Controller Auth

(internal/adapters/controllers/auth_controller.go)

Go



```

package controllers

import (
    "attendance-api/internal/usecases"
    "net/http"

    "github.com/gin-gonic/gin"
)

type AuthController struct {
    authUseCase usecases.AuthUseCase
}

func NewAuthController(authUseCase usecases.AuthUseCase) *AuthController {
    return &AuthController{authUseCase: authUseCase}
}

type LoginRequest struct {
    Email    string `json:"email" binding:"required,email"`
    Password string `json:"password" binding:"required"`
}

type RefreshTokenRequest struct {
}

```

```

    RefreshToken string `json:"refreshToken" binding:"required"`
}

type ForgotPasswordRequest struct {
    Email string `json:"email" binding:"required,email"`
}

type ResetPasswordRequest struct {
    Token          string `json:"token" binding:"required"`
    NewPassword    string `json:"newPassword" binding:"required,min=8"`
}

func (ctrl *AuthController) Login(c *gin.Context) {
    var req LoginRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    accessToken, refreshToken, err := ctrl.authUseCase.Login(req.Email, req.Password)
    if err != nil {
        c.JSON(http.StatusUnauthorized, gin.H{"error": err.Error()})
        return
    }

    c.JSON(http.StatusOK, gin.H{
        "accessToken": accessToken,
        "refreshToken": refreshToken,
        "tokenType":   "Bearer",
    })
}

func (ctrl *AuthController) RefreshToken(c *gin.Context) {
    var req RefreshTokenRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    accToken, refToken, err := ctrl.authUseCase.RefreshToken(req.RefreshToken, c)
    if err != nil {
        c.JSON(http.StatusUnauthorized, gin.H{"error": err.Error()})
        return
    }

    c.JSON(http.StatusOK, gin.H{
        "accessToken": accToken,
        "refreshToken": refToken,
    })
}

```



```

}

func (ctrl *AuthController) Logout(c *gin.Context) {
    sessionID := c.GetString("sessionID")
    if err := ctrl.authUseCase.Logout(sessionID); err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Gagal logout"})
        return
    }
    c.JSON(http.StatusOK, gin.H{"message": "Berhasil logout"})
}

func (ctrl *AuthController) LogoutAll(c *gin.Context) {
    userID := c.GetString("userID")
    if err := ctrl.authUseCase.LogoutAll(userID); err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Gagal logout s
        return
    }
    c.JSON(http.StatusOK, gin.H{"message": "Berhasil logout dari seluruh perangkat
}

func (ctrl *AuthController) GetMe(c *gin.Context) {
    userID := c.GetString("userID")
    user, err := ctrl.authUseCase.GetProfile(userID)
    if err != nil {
        c.JSON(http.StatusNotFound, gin.H{"error": "User tidak ditemukan"})
        return
    }
    c.JSON(http.StatusOK, gin.H{"data": user})
}

func (ctrl *AuthController) ForgotPassword(c *gin.Context) {
    var req ForgotPasswordRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    _ = ctrl.authUseCase.ForgotPassword(req.Email, c.ClientIP(), c.Request.UserAg
    c.JSON(http.StatusOK, gin.H{"message": "Jika email terdaftar, link/token inst
}

func (ctrl *AuthController) ResetPassword(c *gin.Context) {
    var req ResetPasswordRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    if err := ctrl.authUseCase.ResetPassword(req.Token, req.NewPassword); err !=

```

```

        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    c.JSON(http.StatusOK, gin.H{"message": "Password berhasil diperbarui. Silakan

```

8. Update AutoMigration (migrations/migration.go)

Tambahkan entity baru Anda di fungsi auto-migrate:

Go



```

package migrations

import (
    "attendance-api/internal/entities"
    "gorm.io/gorm"
)

func Migrate(db *gorm.DB) error {
    return db.AutoMigrate(
        &entities.User{},
        &entities.UserSession{},
        &entities.AuditLog{},
        // Tambahkan struct entity existing Anda di sini...
    )
}

```

9. Integrasi ke cmd/app/app.go

Berikut penyesuaian file `app.go` milik Anda untuk menggabungkan Middleware, Endpoints Auth, serta Proteksi Endpoint dengan RBAC:

Go



```

package app

import (
    "attendance-api/config"
    "attendance-api/internal/adapters/controllers"
    "attendance-api/internal/adapters/middleware"

```

```

    "attendance-api/internal/adapters/repositories"
    "attendance-api/internal/enums"
    "attendance-api/internal/usecases"
    "attendance-api/migrations"
    "attendance-api/pkg/constants"
    "attendance-api/pkg/drivers/sql"
    "fmt"
    "log"

    "github.com/gin-gonic/gin"
    "gorm.io/gorm"
)

type App struct {
    Db      *gorm.DB
    Router  *gin.Engine
}

func (app *App) ConnectDb() {
    db := sql.OpenDbConnection()
    app.Db = db
    err := migrations.Migrate(db)

    if err != nil {
        log.Fatal("Migrations Failed : -> ", err)
    }
}

func (app *App) Routes() {
    router := gin.Default()
    baseUrl := fmt.Sprintf("%s/v%d", constants.ApiPrevix, constants.ApiVersion)

    // Global Middleware Keamanan
    router.Use(middleware.RateLimiter())

    // --- Injection Module Auth ---
    authRepo := repositories.NewAuthRepository(app.Db)
    authUseCase := usecases.NewAuthUseCase(authRepo)
    authController := controllers.NewAuthController(authUseCase)

    // --- Group Auth (Public Endpoints) ---
    authRoutes := router.Group(fmt.Sprintf("%s/auth", baseUrl))
    {
        authRoutes.POST("/login", authController.Login)
        authRoutes.POST("/refresh-token", authController.RefreshToken)
        authRoutes.POST("/forgot-password", authController.ForgotPassword)
        authRoutes.POST("/reset-password", authController.ResetPassword)
    }
}

```

```

// --- Group Routes Terproteksi (Authentikasi Wajib) ---
protected := router.Group(baseUrl)
protected.Use(middleware.AuthMiddleware(authRepo))
{
    // Authenticated Users (/me, Logout)
    protected.GET("/me", authController.GetMe)
    protected.POST("/logout", authController.Logout)
    protected.POST("/logout-all", authController.LogoutAll)

    // --- RBAC Example: Super Admin Only ---
    adminRoutes := protected.Group("/admin")
    adminRoutes.Use(middleware.RequireRoles(enums.SuperAdmin))
    {
        // Contoh: Tambahkan endpoint spesifik admin di sini
    }

    // --- Group Existing User Routes ---
    userRepo := repositories.NewUserRepository(app.Db)
    userUseCase := usecases.NewUserService(userRepo)
    userController := controllers.NewUserController(userUseCase)

    userRoutes := protected.Group("/user")
    userRoutes.POST("/createUser", middleware.RequireRoles(enums.SuperAdmin))
    userRoutes.GET("/findUserById:id", userController.FindUserById)

    // --- Group Existing School Routes ---
    schoolRepo := repositories.NewSchoolRepository(app.Db)
    schoolUseCase := usecases.NewSchoolService(schoolRepo)
    schoolController := controllers.NewSchoolController(schoolUseCase)

    schoolRoutes := protected.Group("/school")
    schoolRoutes.POST("/createSchool", middleware.RequireRoles(enums.SuperAdmin))
    schoolRoutes.GET("/findSchoolById:id", schoolController.FindSchoolById)

    // --- Group Existing Teacher Routes ---
    teacherRepo := repositories.NewTeacherRepository(app.Db)
    teacherUseCase := usecases.NewTeacherService(teacherRepo, userUseCase)
    teacherController := controllers.NewTeacherController(teacherUseCase)

    teacherRoutes := protected.Group("/teacher")
    teacherRoutes.POST("/createTeacher", middleware.RequireRoles(enums.SuperAdmin))
    teacherRoutes.GET("/findTeacherById:id", middleware.RequireRoles(enums.SuperAdmin))

    // --- Group Existing Grade / Student / AcademicYear / Email Routes ---
    // Lakukan pola penutupan yang sama menggunakan `protected.Group(...)`
}

app.Router = router

```

```
func (app *App) Run() {  
    port := fmt.Sprintf(":%s", config.Config("PORT"))  
    err := app.Router.Run(port)  
    if err != nil {  
        return  
    }  
}
```

10. Konfigurasi Environment (`.env`)

Pastikan file `.env` Anda memuat variabel konfigurasi berikut:

Cuplikan kode



```
PORT=8080  
JWT_SECRET=super-secret-key-enterprise-perbankan-ganti-di-prod  
JWT_TTL_MINUTES=15  
REFRESH_TOKEN_TTL_DAYS=7  
MAX_LOGIN_ATTEMPTS=5
```

 Pro sedang banyak diminta

Model lain digunakan untuk respons ini. Hal ini tidak mengurangi batas penyimpanan Anda.